

BRACT's

Vishwakarma Institute of Technology, Pune

Practical Implementation Sheet

Department: CSE-AI Semester: 5 Academic Year: 2026-27

Division: C Batch: 2 Practical No: 1

Roll no: 62 PRN No: 12411517 Name of Student: Akshada Lohakare

Subject Name : Deep Learning

Problem Statement : Install and configure TensorFlow/Keras in Google Colab. Perform data preprocessing, normalization, train-test splitting, and data visualization on a sample dataset.

Algorithm :

1. Start the program.
2. Import the required libraries such as TensorFlow, Keras, NumPy, Matplotlib, and Scikit-learn.
3. Load the Fashion MNIST dataset using Keras.
4. Split the dataset into training and testing datasets.
5. Visualize sample images from the dataset to understand the data distribution.
6. Normalize the image pixel values from the range 0–255 to 0–1 by dividing each pixel value by 255.0.
7. Split the training dataset further into training and validation datasets using `train_test_split()`.
8. Create a neural network model using the Sequential API.
9. Add a Flatten layer to convert the 28×28 images into a one-dimensional vector.
10. Add a Dense hidden layer with 128 neurons and ReLU activation function.
11. Add an output layer with 10 neurons and Softmax activation function for classification.
12. Compile the model using the Adam optimizer, Sparse Categorical Crossentropy loss function, and Accuracy metric.
13. Train the model using the training dataset for a specified number of epochs.
14. Evaluate the trained model using the testing dataset.
15. Display the model accuracy and loss values.
16. Save the trained model for future use.

Code :

```
1. !pip install tensorflow
```

2. Importing libraries:

```
import tensorflow as tf
import matplotlib.pyplot as plt
```

```
3. # Get data and split into train and test datasets
(train_images, train_labels), (test_images, test_labels) =
tf.keras.datasets.mnist.load_data()
```

4. Scale down values of pixels from 0-255 to 0-1

```
train_images = train_images / 255.0
test_images = test_images / 255.0
```

5. Visualizing data

```
print(train_images.shape)
print(test_images.shape)
print(train_labels)
```

6. Display first image

```
plt.imshow(train_images[1], cmap='gray')
plt.show()
```

7. Defining neural network model

```
my_model = tf.keras.models.Sequential()
my_model.add(tf.keras.layers.Flatten(input_shape=(28, 28)))
my_model.add(tf.keras.layers.Dense(128, activation='relu'))
my_model.add(tf.keras.layers.Dense(10, activation='softmax'))
```

8. Compiling the model

```
my_model.compile(optimizer='adam',
loss='sparse_categorical_crossentropy', metrics=['accuracy'])
```

9. Training the model

```
my_model.fit(train_images, train_labels, epochs=5)
```

10. Checking model accuracy on test data

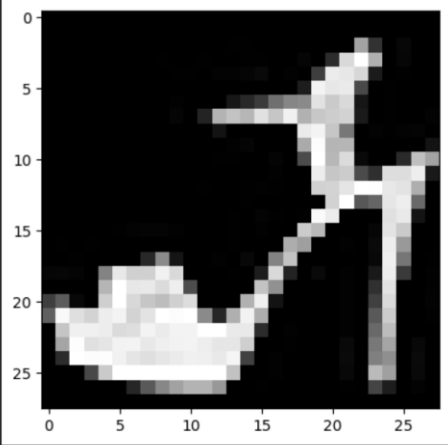
```
val_loss, val_acc = my_model.evaluate(test_images, test_labels)
print('Test accuracy: ', val_acc)
```

11. Saving the model

```
my_model.save("my_mnist_model.keras")
```

Output :

```
# Display first image
plt.imshow(train_images[9], cmap='gray')
plt.show()
```



```
# training the model
my_model.fit(train_images, train_labels, epochs=5)

Epoch 1/5
1875/1875 ————— 9s 4ms/step - accuracy: 0.8246 - loss: 0.4988
Epoch 2/5
1875/1875 ————— 8s 4ms/step - accuracy: 0.8637 - loss: 0.3752
Epoch 3/5
1875/1875 ————— 9s 5ms/step - accuracy: 0.8755 - loss: 0.3376
Epoch 4/5
1875/1875 ————— 8s 4ms/step - accuracy: 0.8857 - loss: 0.3123
Epoch 5/5
1875/1875 ————— 9s 5ms/step - accuracy: 0.8908 - loss: 0.2969
<keras.src.callbacks.history.History at 0x7d2c305581d0>
```